

CRIMSON ORBIT – MUSICAL BAND

Complete Course Project Presentation Script & Teacher Q&A; Defense

Target: Bare-Metal 8086 Assembly Language | **Rule:** Each Member Speaks Exactly Once

Speaker	Assigned Roles & Contributions	Timing
1. AKASH	Intro, Architecture Research, Piano Module & Art, System Assembly	~2.0 min
2. SANSKAR	Guitar Module: Fretboard, Strings, Vibrato & Chord Strumming	~1.5 min
3. KRISHNA	Drums Module: 7-Piece Kit, 16-Bit LFSR White Noise & Percussion	~1.5 min
4. HARI	Song Repertoire Data: 6 Scores, Word Encoding & High-Hz Tuning	~1.0 min
5. GHANSHAM	Demo Player Engine, 8-Channel Visualizer, Non-Blocking ESC & Conclusion	~1.5 min

1. AKASH

PROJECT LEAD — INTRODUCTION, HARDWARE RESEARCH, PIANO MODULE & SYSTEM ASSEMBLY

■ Complete Speaking Script (Speak Once):

"Good morning/afternoon, Respected Professor and classmates. Our course project is **Crimson Orbit – Musical Band**, an interactive bare-metal audio workstation programmed entirely in **8086 Assembly Language**.

In our project, I handled the **system architecture research**: discovering how to produce authentic sound and graphics directly on bare-metal hardware without DOS, Windows, or external drivers. I researched the **Intel 8254 Programmable Interval Timer (PIT)** at Ports 42h and 43h, and the **8255 PPI** at Port 61h. We divide the PIT base clock frequency of 1,193,180 Hz by our target note frequency to obtain a 16-bit divisor, which is written LSB-then-MSB to Channel 2 to physically vibrate the PC speaker diaphragm.

Next, I designed and coded the **Interactive Concert Piano module (piano.asm)**. I created the 3D beveled keyboard interface using CP437 graphical characters, featuring 8 ivory white keys and 5 ebony black keys with active depression depth and shadow relief when struck. To eliminate muddy low-frequency speaker hum, I standardized the tuning to **Concert Octave 5 (523 Hz to 1046 Hz)**, built a live **[TAB] Octave Switcher** for Octaves 4, 5, and 6, physically modeled the felt hammer strike with a 12ms attack overtone, and programmed a **[SPACE] Sustain Pedal** for extended acoustic resonance.

Finally, I handled **System Integration & Assembly**: I wrote the MBR bootloader (boot.asm) that boots from Sector 1 and uses BIOS INT 13h, AH=02h to load 24 sectors into memory at segment 1000h:0000h, assembled all team modules into kernel.asm, and built our bootable 1.44 MB floppy image (new2.flp).

I will now pass the presentation to Sanskar to demonstrate the Guitar module."

■ Rapid-Fire Teacher Defense (When Asked 'What did you do?'):

Teacher: "Akash, what was your specific contribution to this project?"

Your Answer: "Sir/Ma'am, I handled three core areas: **First**, architecture research on the Intel 8254 PIT (Ports 42h/43h) and 8255 PPI (Port 61h) to drive the PC speaker. **Second**, I developed the Piano module (piano.asm), building the 3D ivory/ebony keyboard art, concert Octave 5 frequencies, hammer attack transient, and sustain pedal. **Third**, I assembled the entire system—writing boot.asm with INT 13h floppy loading, linking all modular asm files in kernel.asm, and producing the bootable floppy disk image."

2. SANSKAR

GUITAR MODULE & STRING PHYSICAL MODELING

■ Complete Speaking Script (Speak Once):

"Thank you, Akash. I designed and implemented the **Simulated Acoustic & Lead Guitar module (guitar.asm)**.

My goal was to capture the organic, singing quality of a real guitar string rather than a flat electronic beep. I mapped the 6 strings to **Concert Lead Tuning**: High E at 659 Hz down to Low E at 165 Hz, and added an on-the-fly **[TAB] Mode Switch** that activates high-register Lead Solo Shred mode up to 1318 Hz.

On the sound synthesis side in speaker.asm, I programmed physical modeling for the **plectrum pick attack transient**: when any string is plucked using keys Q through Y, it immediately triggers a +20% frequency chirp for 14ms simulating the pick scraping the wire, settling into a **singing finger vibrato** that oscillates the pitch by ± 3 Hz in continuous sinusoidal cycles.

Visually, I drew the 12-fret guitar neck with heavy wound bronze bass strings using double-lines (■), silver steel treble strings (■), and authentic inlay fret markers at positions 3, 5, 7, 9, and 12. I also implemented the **[S] Chord Strumming feature**, which arpeggiates across full chords like E minor, G, C, and D with realistic string vibration wave animations (~~~~).

Now, Krishna will present the Drums and percussion synthesis."

■ Rapid-Fire Teacher Defense (When Asked 'What did you do?'):

Teacher: "Sanskar, what was your specific contribution?"

Your Answer: "Sir/Ma'am, I developed the Guitar module (guitar.asm). I built the visual 12-fret fretboard with wound-bronze and steel string graphics and fret inlay markers. In the audio engine, I synthesized the **plectrum attack transient** (+20% frequency chirp for 14ms) and **singing finger vibrato** (± 3 Hz sinusoidal pitch modulation). I also developed the **[S] Chord Strumming engine** that sweeps across multi-string chords with live wave vibration animations."

3. KRISHNA

DRUMS MODULE & LFSR WHITE NOISE SYNTHESIS

■ Complete Speaking Script (Speak Once):

"Thank you, Sanskar. I developed the **Studio Drum Kit module (drums.asm)** and percussion synthesis engine.

Percussion is the biggest challenge on an 8086 PC speaker because the hardware is strictly monophonic and designed only for square waves. Real drums require explosive broadband white noise and rapid acoustic pitch glides.

To achieve this, I implemented a **16-bit Galois Linear Feedback Shift Register (LFSR)** in assembly (Sound_RandomLFSR). By shifting a seed register and applying polynomial feedback mask 0B400h, we generate hardware-speed pseudo-random toggles on Port 61h, producing genuine acoustic white noise for:

1. **Acoustic Snare Drum [W]**: A dual-layer sound combining snappy LFSR wire rattle with a resonant wooden shell pop decaying from 240 Hz to 130 Hz.
2. **Closed Hi-Hat [E] & Open Hi-Hat [Y]**: A tight 10ms metallic choke for closed, and a 60ms shimmering sizzle decay for open.
3. **Crash Cymbal [T] & Ride Bell [U]**: An explosive white-noise burst with 280ms exponential decay, and a 1318 Hz brass bell ping.
4. **22-inch Bass Kick Drum [Q]**: A high-impact beater strike click (420 Hz -> 280 Hz) dropping into a resonant bass shell thump down to 42 Hz.

Visually, I created a stage layout with circular drum rims and dynamic radial shockwave impact animations (((■))) with strike intensity meters.

Now, Hari will explain our Pre-Installed Songs Repertoire."

■ Rapid-Fire Teacher Defense (When Asked 'What did you do?'):

Teacher: "Krishna, what was your specific contribution?"

Your Answer: "Sir/Ma'am, I developed the Drum Kit module (drums.asm). Because PC speaker hardware only generates single tones, I wrote a **16-bit Galois LFSR pseudo-random noise generator** in assembly using polynomial mask 0B400h to generate white noise. With this, I synthesized the dual-layer acoustic snare, closed/open hi-hats, 280ms crash cymbal splash, and a punchy 22" kick drum glide."

4. HARI

SONG REPERTOIRE DATA ARCHITECTURE & HIGH-HERTZ TUNING

■ Complete Speaking Script (Speak Once):

"Thank you, Krishna. Together with Ghansham, we built the Demo Songs and Visualizer system. My dedicated responsibility was the **Song Repertoire Data Architecture (songs.asm)**.

I musically transcribed and encoded 6 complete classical and traditional compositions into 8086 memory:

1. *Happy Birthday*
2. *Twinkle Twinkle Little Star*
3. *Ode to Joy (Beethoven's 9th Symphony)*
4. *Jingle Bells*
5. *Mary Had a Little Lamb*
6. *London Bridge Is Falling Down*

Standard Octave 4 notes on PC speaker sound sluggish and buzzy. To make the songs sound sweet and professional, I transposed every single score into **Concert Octaves 5 and 6**, elevating note frequencies from **523 Hz up to 1568 Hz**.

Each song score is structured in memory as a compact array of paired 16-bit words: the **Frequency in Hertz** followed by the **Duration in Milliseconds**, terminated by the sentinel value 0FFFFh (SONG_END). This allows zero-latency score streaming and instantaneous memory access.

I will now hand over to Ghansham to explain the Demo Player and Visualizer."

■ Rapid-Fire Teacher Defense (When Asked 'What did you do?'):

Teacher: "Hari, what was your specific contribution?"

Your Answer: "Sir/Ma'am, I was responsible for the Song Repertoire Data (songs.asm). I transcribed and encoded 6 complete musical scores into 8086 memory as paired 16-bit words of frequency and duration. I also **transposed all songs into Concert Octaves 5 and 6 (523 Hz to 1568 Hz)**, eliminating low-frequency square-wave buzzing so all melodies play in clear, sweet, singing harmony."

5. GHANSHAM

DEMO PLAYER ENGINE, AUDIO VISUALIZER & CONCLUSION

■ Complete Speaking Script (Speak Once):

"Thank you, Hari. I developed the **Demo Player Engine and Real-Time Audio Visualizer (demo.asm)**, and I will conclude our presentation.

When a user selects any of Hari's 6 songs, my player prompts them to choose an **Instrument Style**:

- **Piano Style**: Routes playback through the felt hammer strike and decay envelope.
- **Guitar Style**: Routes the melody through the plectrum attack and singing vibrato.
- **Drums Style**: Translates melodic pitches into a rhythmic acoustic percussion groove—low notes trigger the bass kick, mid notes trigger the LFSR snare, high notes trigger the tom, peak notes trigger the crash cymbal, and rests trigger the hi-hat.

During playback, my engine renders a live **8-column Spectrum Equalizer Visualizer** mapped from C5 to C6. As notes play, the corresponding frequency bars bounce dynamically with peak-hold lighting, accompanied by a real-time Hertz frequency readout.

I also built a non-blocking keyboard polling loop using BIOS INT 16h, AH=01h so the user can cleanly interrupt any song with **[ESC]** without crashing or freezing the CPU.

Conclusion: Our team proved that with pure 8086 assembly language and direct hardware control, we can build a fully functional, rich, multi-instrument musical workstation on bare metal. We are now ready for our live demonstration and to answer your questions. Thank you!"

■ Rapid-Fire Teacher Defense (When Asked 'What did you do?'):

Teacher: "Ghansham, what was your specific contribution?"

Your Answer: "Sir/Ma'am, I developed the Demo Player Engine and Visualizer (demo.asm). I programmed the multi-instrument interpreter that translates songs into Piano, Guitar, or rhythmic Drum groove styles. I also created the **real-time 8-channel spectrum equalizer visualizer** with dancing bars and live frequency display, and implemented non-blocking BIOS INT 16h keyboard polling for safe **[ESC]** interruption."